FOUNDATIONS FOR AI
A New Take on the AI Gateway
Join us Nov 5 for a new approach to the AI gateway: connected systems, governed access, lower cost.
Sign Up Today

ARTIFICIAL INTELLIGENCE

How I Fine-Tuned Granite-Vision 2B to Beat a 90B Model — Insights and Lessons Learned

A hands-on journey exploring fine-tuning techniques that unlock the power of small vision models.

Julio Sanchez
July 25, 2025 • 20 min read

Home
About
Projects
Contact
IML
Hello World!
<H1>
<body>

Image generated by the author with ideogram.ai

Fine-tuning large language or vision-language models is a powerful technique that unlocks their potential on specialized tasks. However, despite their effectiveness, these approaches are often out of reach for many users due to their high computational cost and the need for GPUs with large VRAM — resources that only a small percentage of end users can access.

In this project, I fine-tuned IBM's Granite-Vision 2B, a relatively small yet powerful vision-language model, to tackle the challenge of converting images of tables into clean, structured HTML code.

What makes this project particularly exciting is that the fine-tuning was performed on a consumer-grade GPU — the NVIDIA RTX 4070 Ti Super — and yet, the resulting 2-billion-parameter model was able to outperform much larger models, including meta-llama/Llama-3.2–90B-Vision, on this image-to-text generation task. This success not only demonstrates the power of parameter-efficient fine-tuning methods like LoRA but also highlights the practical value of building specialized small models tailored to specific problems.

In this post, I'll walk you through the motivation behind this work, the model and dataset choices, the custom HTML similarity metric I adapted, the experiments and results, and finally, the key insights and lessons learned throughout the process. Whether you're interested in vision-language models, fine-tuning techniques, or practical AI applications, I hope this journey offers useful takeaways. The fine-tuning code used for this project was adapted from HuggingFace's Granite Vision fine-tuning cookbook, authored by Eli Schwartz, who in turn adapted the original code from Sergio Paniego.

Learn this step by step with the interactive **AI Engineer** roadmap.

Motivation

While working on Retrieval-Augmented Generation (RAG) projects, I encountered a major challenge: accurately extracting large and complex tables from PDFs, especially when these tables appeared as images. Despite trying different approaches — including tools like Unstructured and large vision-language models such as Meta's Llama 90B — the results often fell short of the accuracy needed.

This led me to consider a different approach: a **small, specialized vision-language model focused exclusively on table understanding and extraction**. Such a model could serve as a dedicated preprocessing step to significantly improve RAG pipelines that rely on accurate table extraction.

Around the same time, IBM released Granite-Vision 2B — a vision-language model with just the right balance of size and power. It's capable enough to handle complex tables, yet small enough to be fine-tuned on consumer-grade GPUs with 16 GB of VRAM. This made it an ideal candidate for my project.

The Task: Image to HTML (Table Extraction)

One important design choice was the target format: **HTML**. By converting tables into clean HTML code, we obtain a structured and widely supported representation that can be easily converted into other formats. For example, HTML tables can be readily imported into data analysis tools like Pandas as dataframes, making downstream processing and analysis much more efficient.

The original plan was to build a custom dataset by extracting HTML table tags, rendering them as images, and pairing each image with its corresponding HTML code. Fortunately, I found a solution: the PubTabNet-HTML dataset, which includes over 568,000 image–HTML pairs, far more than needed for this project.

PubTabNet was developed by IBM and is based on scientific articles from the **PubMed Central Open Access Subset (commercial use collection)**. The tables were extracted by aligning PDF and XML versions of the articles. The **annotations** (i.e., the HTML labels) are licensed under the **Community Data License Agreement – Permissive – Version 1.0**, and while **IBM does not own the images**, they are used in accordance with the PMC Open Access Subset Terms of Use. This makes the dataset suitable for both research and commercial applications, provided the license terms are followed.

Custom Metric: HTML Similarity

Standard text similarity metrics like BLEU or ROUGE are insufficient for evaluating HTML table generation because they primarily focus on surface-level text matching and ignore important structural and stylistic aspects of HTML code.

To better capture the quality of generated HTML tables, I adapted a custom **HTML Similarity** metric that combines multiple complementary components, where the most important ones (style and structure) are imported from **niteru**:

- **Style similarity (S):** Extracts CSS classes of each html document and calculates the jaccard similarity of the sets of classes.
- **Structural similarity (T):** Uses sequence comparison of the html tags to compute the similarity.
- **Content similarity (C):** Based on normalized edit distance between the extracted plain text content of the tables.
- **Token overlap similarity (J):** The Jaccard similarity between the sets of content tokens.

The final similarity score **M** is a weighted sum of these components:

$$M = 0.10 \times S + 0.40 \times T + 0.30 \times C + 0.20 \times J$$

where each component $S, T, C, J \in [0, 1]$, and so $M \in [0, 1]$.

I manually tested the metric on various example outputs, iteratively adjusting the weighting coefficients to better capture meaningful similarities. This process resulted in a balanced evaluation that fairly rewards accurate table structure and style, alongside precise textual content. Python implementation is as follows:

```python
from torchmetrics.text import EditDistance
from niteru import style_similarity, structural_similarity

ed_distance = EditDistance()

def extract_table_text(html):
    """Extracts only the text from an HTML table in row-wise space-separated format."""
    soup = BeautifulSoup(html, "html.parser")
    table = soup.find("table")  # Find the first table
    if not table:
        return ""
    # Extract rows and join cells with spaces
    return "\n".join(" ".join(cell.get_text(strip=True) for cell in row.find_all(["th",

def extract_html_table(html):
    """Extracts html table from text"""
    match = re.search(r'<table\b.*?</table>', html, re.DOTALL | re.IGNORECASE)
    if match:
        table_html = match.group()
        return table_html
    else:
        return html

def html_similarity(html1, html2):
    html1 = extract_html_table(html1)
    html2 = extract_html_table(html2)
    # Compute individual similarity scores
    style_sim = style_similarity(html1, html2)  # Assume returns [0,1]
    struct_sim = structural_similarity(html1, html2)  # Assume returns [0,1]
    txt1, txt2 = extract_table_text(html1), extract_table_text(html2)
    content_sim = 1 - (ed_distance(txt1, txt2) /
                               max(len(txt1), len(txt2) + 1e-10))  # Avoid division
    jaccard_sim = 1 - (len(set(txt1.split()).intersection(set(txt2.split()))) /
                          len(set(txt1.split()).union(set(txt2.split()))) + 1e-10)

    # Weighted sum of the similarities
    final_score = (0.10 * style_sim) + (0.40 * struct_sim) + (0.30 * content_sim) + (0.
    # Ensure final score is in [0,1]
    final_score = max(0, min(1, final_score))
    return final_score
```

The metric also includes a regex-based function to extract only the HTML content within `<table>` tags. This was necessary because one of the reference models only generated incomplete or extra HTML outside of the table structure. By focusing the comparison strictly on the table content, the metric provides a more fair and meaningful evaluation across models.

Developing a custom evaluation metric like this is crucial for reliably tracking model improvements and benchmarking performance against reference models.

Training Setup

To fine-tune the model efficiently on my NVIDIA RTX 4070 Ti Super, which has 16 GB VRAM, I used **LoRA** (Low-Rank Adaptation). This allowed me to update only a small number of parameters, significantly reducing GPU memory usage. In fact, during training, the model used only about half of the available VRAM — with enough headroom to play around with longer sequences, but not enough to handle more than one batch. Additionally, LoRA is generally faster to train than approaches like QLoRA.

LoRA Setup

I used the following LoRA configuration:

```python
# Setup LoRA
target_modules = []
for layer_type in layers_to_tune:
    target_modules.extend(
        name for name, _ in model.named_modules()
        if (layer_type in name)
        and '_proj' in name
    )
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    lora_dropout=0.1,
    target_modules=target_modules,
    use_dora=True,
    init_lora_weights="gaussian"
)
```

Key points:

- `r=16` : This low-rank dimension provides a good balance between model capacity and GPU memory usage.
- `use_dora=True` : DoRA (Weight-Decomposed Low Rank Adaptation) improves the learning capacity and stability of LoRA by decomposing the pretrained weights into magnitude and direction components, helping the model better resemble the capacity of full fine-tuning — all without adding inference overhead. Performed slightly better than the default setting.
- `init_lora_weights="gaussian"` : No particular reason, I didn't want to experiment with this parameter.
- `target_modules` : This flexible setup allows selectively targeting vision layers, language layers, or both, depending on the experiment. In practice, vision layers remained unaffected — even with `use_dora=False` — since DoRA currently supports only embedding, linear, and Conv2d layers. As a result, I fine-tuned only the language layers.

Dataset Setup

During my initial experiments, I kept running into **out-of-memory (OOM)** errors — even though there was still plenty of available GPU VRAM after loading model, LoRA layers and optimizer parameters (around 4GB still free). There were no memory spikes during training, but the crashes consistently happened at the same training step.

After some investigation, I realized that the problem was caused by large tables, which resulted in extremely long token sequences. To address this, I adjusted the `max_seq_length` parameter and filtered out samples that exceeded this limit. After experimentation, I found that using `max_seq_length = 1024` allowed me to fine-tune the model reliably without triggering OOM errors.

To filter out oversized tables, I wrote a simple data processing function that:

- Filters out samples whose HTML token length exceeds `max_seq_length`
- Automatically balances the number of training and test samples
- Uses **streaming** to avoid loading the entire dataset into memory (PubTabNet-HTML is quite large, around 10 GB on disk)

.

```python
def load_process_filter_dataset(dataset, max_seq_length, num_train_images, num_test_ima
    global processor
    ds = load_dataset(dataset, split='train', streaming=True)
    max_html_tokens = max_seq_length - len(processor.tokenizer.tokenize(system_message)
    num_total_needed = num_train_images + num_test_images
    filtered_samples = []
    p_bar = tqdm(total=num_total_needed, desc="Filtering dataset samples")
    for sample in ds:
        processed = process_and_filter_example(sample, max_html_tokens)
        if processed:
            filtered_samples.append(processed)
            p_bar.update(1)
        if len(filtered_samples) >= num_total_needed:
            break
    p_bar.close()
    # Convert to in-memory dataset
    ds_filtered = Dataset.from_list(filtered_samples)
    # Split into train/test
    ds_train = ds_filtered.select(range(num_train_images))
    ds_test = ds_filtered.select(range(num_train_images, num_total_needed))
    return ds_train, ds_test

def process_and_filter_example(example, max_html_tokens):
    global processor
    extracted_table = extract_html_table(example['html_table'])
    token_count = len(processor.tokenizer.tokenize(extracted_table))
    if token_count < max_html_tokens:
        example['html_table'] = extracted_table
        return example
    return None
```

The final configuration included `num_train_images=10000` and `num_test_images=250` to compute the evaluation loss.

Fine-Tuning Configuration

For training, I used the **Transformers SFTTrainer** to fine-tune the model:

```python
# Training arguments
    training_args = SFTConfig(
        output_dir=f"src/models/{model_name.split('/')[-1].replace('-', '_', 1).split('
        num_train_epochs=1,
        per_device_train_batch_size=1,
        per_device_eval_batch_size=1,
        gradient_accumulation_steps=gradient_accumulation_steps,
        max_seq_length=max_seq_length,
        warmup_steps=10,
        learning_rate=3e-4,
        weight_decay=0.01,
        logging_strategy="steps",
        eval_strategy='steps',
        logging_steps=25,
        save_strategy="steps",
        save_steps=50,
        save_total_limit=1,
        greater_is_better=False,
        load_best_model_at_end=True,
        optim="adamw_torch_fused",
        bf16=True,
        push_to_hub=False,
        report_to="wandb" if not debug else "none",
        remove_unused_columns=False,
        gradient_checkpointing=True,
        dataset_text_field="",
        dataset_kwargs={"skip_prepare_dataset": True},
        dataset_num_proc=8
    )

# Setup Trainer
    trainer = SFTTrainer(
        model=model,
        args=training_args,
        train_dataset=train_dataset,
        eval_dataset=test_dataset,
        data_collator=collate_fn,
        peft_config=peft_config,
        processing_class=processor.tokenizer
    )
```

Key points:

- `num_train_epochs=1` : The dataset is very large, and to run multiple experiments efficiently, I chose to train for only one full epoch while maximizing learning per sample and number of training samples.
- `per_device_train_batch_size=1` : Larger batch sizes would not fit in GPU memory without significantly reducing max_seq_length — which would hurt performance on large tables. Keeping longer sequences was more important for this task.
- `gradient_accumulation_steps=8` : Used to effectively simulate a larger batch size and help stabilize the learning process, compensating for the small physical batch. This is the final value, but experimented with `gradient_accumulation_steps=4` as well.
- `optim="adamw_torch_fused"` and `bf16=True` : These settings leverage modern NVIDIA architectures (Ada Lovelace) to accelerate training and reduce memory usage — as recommended for this hardware.

Evaluation Loss Workaround

At the time of developing the project, there is a known issue in the Transformers + LoRA integration that causes an error when running evaluation with a validation dataset during training. Fortunately, a community-tested workaround is available (although not yet merged into the main branch), and I successfully used this fix in my experiments.

Evaluation (Inference) Setup

The evaluation dataset used for final scoring was completely independent from the `eval_dataset` used during training. It consists of **500 randomly selected images**, none of which were included in either the `train_dataset` or the training `eval_dataset`.

Once fine-tuning was complete, I used the **best model checkpoint** — selected based on the lowest evaluation loss — to run inference on these 500 samples.

Initially, I attempted to perform inference by simply **loading the LoRA/DoRA adapter** on top of the base model. However, I found that inference with **DoRA adapters is extremely slow** when not merged into the model weights (as explained in the official PEFT docs). In fact, generating one test random sample took about **90 seconds** in this configuration.

To resolve this, I merged the adapter weights into the base model — which is the recommended practice — and after merging, inference speed improved dramatically: **down to ~20 seconds for the same sample**, making full evaluation runs much more practical.

The **reference models** used for comparison with my fine-tuned models are:

- meta-llama/Llama-3.2–90B-Vision: Meta's massive **90-billion parameter**model — the main baseline I aimed to surpass through specialization and parameter-efficient fine-tuning of a much smaller VLM.
- KennethTM/pix2struct-base-table2html: A much smaller model fine-tuned from Google's pix2struct-base, highly specialized for exactly the same dataset I used in this project. Thanks to its smaller size, the developer(s) was able to train it for many more samples and over longer training runs — demonstrating the key advantage of using **smaller, targeted models** for specific tasks.

These two baselines allowed me to benchmark both **scaling-based performance** (vs the 90B model) and **specialization efficiency** (vs the smaller, dedicated Pix2Struct model).

Experiments & Results

A total of 9 experiments were conducted, iteratively modifying one or two components at a time. The goal was to understand the effect of each change on model performance, gradually refining the setup to achieve the best possible **HTML Similarity** score compared to reference models.

The experimental process was incremental: whenever a change improved the results, it was incorporated into the next round of experiments and continued exploring new variations.

The experiments focused on adjusting the following components:

1. **Vision vs. Language Layers**

- 1.1 `lang_only`
- 1.2 `vision_only`
- 1.3 `lang_vision`

2. Ground Truth Output Format

- 2.1 `lang_table_only`

3. Training Framework

- 3.1 `lang_table_unsloth`
- 3.2 `vision_table_unsloth`

4. Gradient Accumulation

- 4.1 `lang_table_only_2`

5. Prompt Format

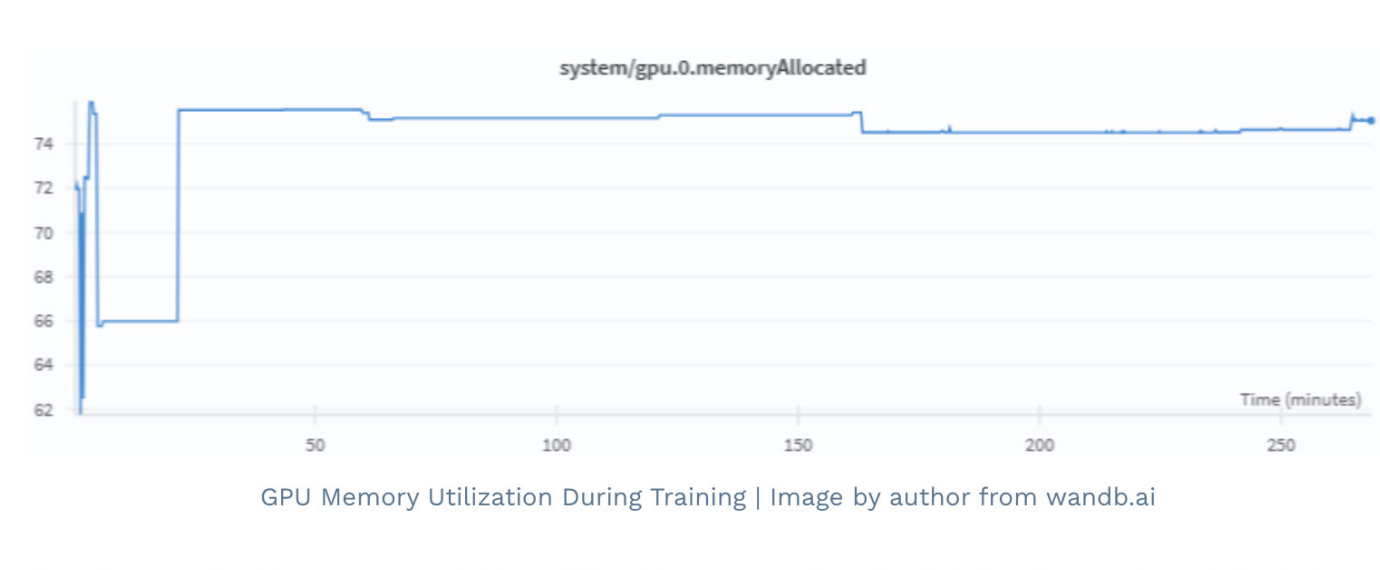
- 5.1 `lang_table_only_3`

6. Gradient Accumulation & Dataset Size

- 6.1 `lang_table_only_4`

Both the **evaluation loss** and the **HTML Similarity** metric were used to assess model performance, and I found them to be well correlated — confirming that HTML Similarity is a good proxy for how well the model is learning the task.

Before diving into the results of each experiment, let's first look at **GPU memory utilization during training**, which is often the most critical factor in determining whether a model can be fine-tuned on consumer hardware.



GPU Memory Utilization During Training | Image by author from wandb.ai

As shown in the graph, GPU utilization remained stable throughout training — averaging around **75% VRAM usage**, or roughly **12 GB** on my GPU. Most of VRAM usage (~5.5 GB) is the frozen model weights. LoRA gradients + optimizer states take very little (<< 1 GB). Activations + overhead should fill the rest (~5–6 GB), which depends on `batch_size` and `max_seq_length`.

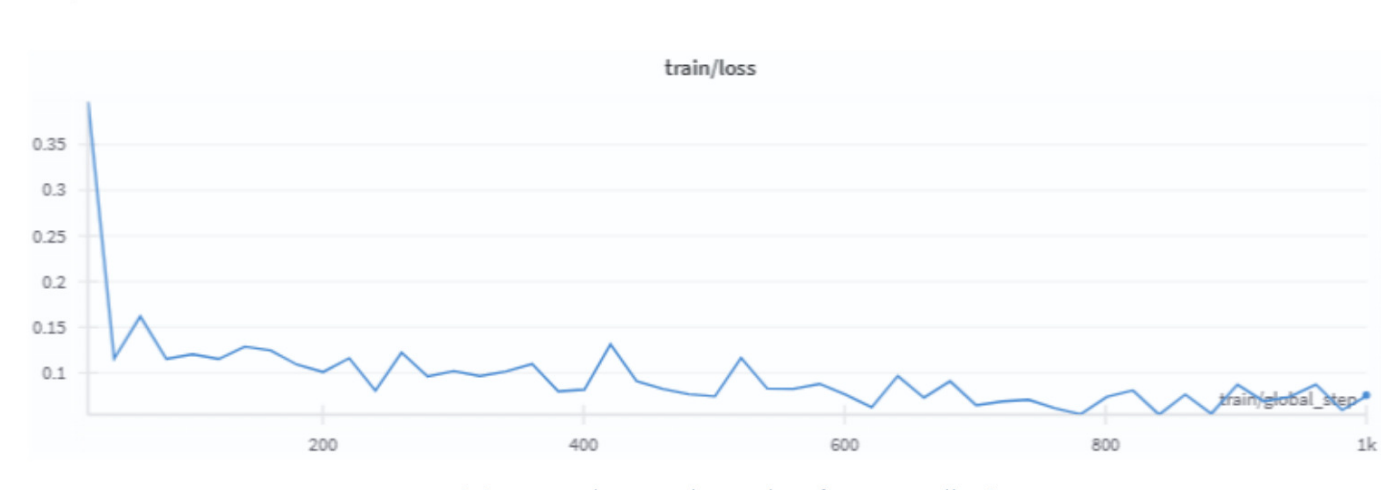
First Run: `lang_only`

This experiment uses the following initial components/parameters:

Parameter	Value
`layers_to_tune`	"language_model"
`gradient_accumulation_steps`	4
`num_train_images`	5000
Prompt	"Convert user's image to HTML"
Output Format	"<html> … </html>"
Training Framework	HuggingFace Transformers

These were the starting values for the first experiment. In subsequent runs, I modified many of them as I refined the approach. This first experiment focused only on tuning **language layers**, while training the model to predict the full raw HTML output — including everything inside and around the `<table>` tags.

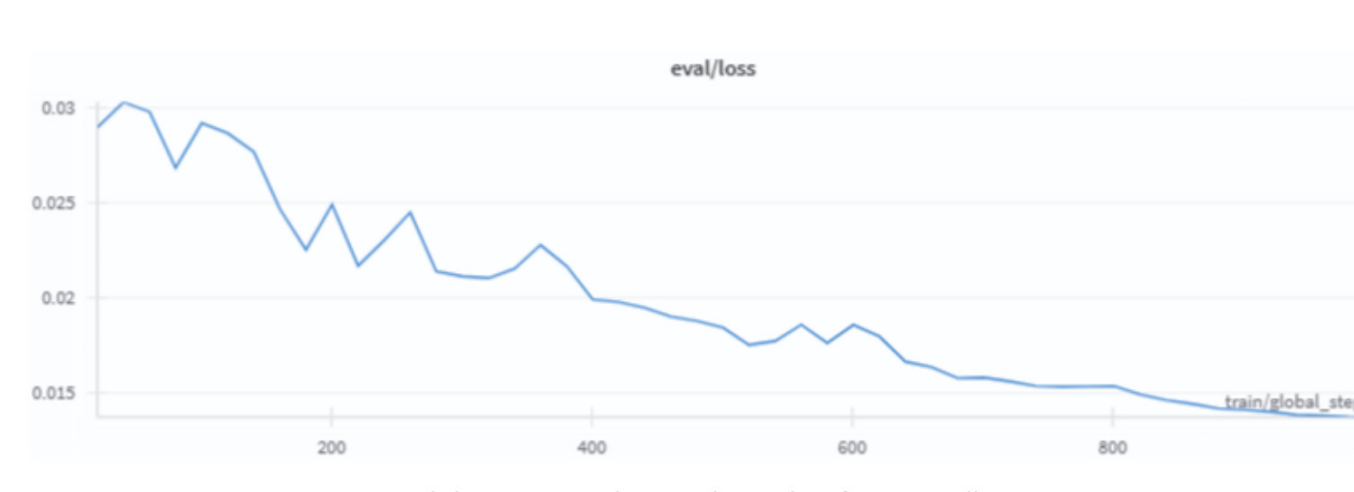
Since this was the first run, I'll include the **training loss curve** here to illustrate how it behaves. For later experiments, I'll omit this graph — as the behavior was similar across runs, with minor variations. In practice, **evaluation loss** is more useful for comparing performance across experiments.



Training Loss | Image by author from wandb.ai

One important note about the logging configuration: `logging_steps=25` means that the training loss is only logged after every 25 steps, where each logged value is the average over `gradient_accumulation_steps=4`. As a result, the largest drop in loss appears at the second log point — where most of the initial learning happens. After that, the model continues learning more gradually, with a slow decreasing trend, depending on the difficulty of the training samples.

Now, let's take a look at the **evaluation loss**:



Validation Loss 1 | Image by author from wandb.ai

Since we are evaluating on the same set of 250 validation samples, the evaluation loss curve gives us a more stable and meaningful view of model learning — and will serve as a baseline for comparisons across future runs.

Here, we observe a clear and consistent downward trend throughout training. The initial loss starts close to **0.03**, with a steady improvement as training progresses, eventually stabilizing just below **0.015**.

The smooth nature of this curve — compared to the more variable training loss — reflects the regular structure of the validation set and confirms that the model is generalizing well to unseen samples, even with a small batch size and a single epoch of training.

Now, let's compare the performance of this fine-tuned model against the